

Task 5: Exploratory Data Analysis (EDA)

Dataset: Retail Store Sales

Tools: Python (Pandas, Numpy, Matplotlib, Seaborn)

Objective: Extract insights using visual and statistical exploration find patterns, trends, and anomalies in the data.

EDA is always the first step after cleaning. Before building any model or dashboard, we need to *understand* our data what it looks like, how values are distributed, and which columns are related to each other.

PART 1: Data Cleaning

Import libraries and load the data

We need two libraries:

- **pandas** — to load and work with our data as a table (called a DataFrame)
- **numpy** — for numeric operations, mainly used when handling missing values

We then load our CSV file into a DataFrame called `df` (short for "dataframe").

```
In [7]: import pandas as pd  
import numpy as np
```

```
In [9]: df = pd.read_csv(r'C:\Users\Admin\Downloads\archive\retail_store_sales.csv')
```

```
In [11]: # Let's look at the first 5 rows to get a feel for the data  
df.head()
```

Out[11]:

	Transaction ID	Customer ID	Category	Item	Price Per Unit	Quantity	Total Spent	Payment Method	Location	Transaction Date	Discount Applied
0	TXN_6867343	CUST_09	Patisserie	Item_10_PAT	18.5	10.0	185.0	Digital Wallet	Online	2024-04-08	True
1	TXN_3731986	CUST_22	Milk Products	Item_17_MILK	29.0	9.0	261.0	Digital Wallet	Online	2023-07-23	True
2	TXN_9303719	CUST_02	Butchers	Item_12_BUT	21.5	2.0	43.0	Credit Card	Online	2022-10-05	False
3	TXN_9458126	CUST_06	Beverages	Item_16_BEV	27.5	9.0	247.5	Credit Card	Online	2022-05-07	NaN
4	TXN_4575373	CUST_05	Food	Item_6_FOOD	12.5	7.0	87.5	Digital Wallet	Online	2022-10-02	False

```
In [13]: # How many rows and columns do we have?
print("Shape of dataset (rows, columns):", df.shape)
```

Shape of dataset (rows, columns): (12575, 11)

Step 1: Identify and handle missing values

Why this matters: Missing values (also called "nulls" or "NaN") can break calculations, cause wrong conclusions, or crash machine learning models. Before fixing anything, we first need to see where the gaps are.

We use `.isnull().sum()` this checks every cell in every column and counts how many are empty.

```
In [15]: # Check for missing values in each column
df.isnull().sum()
```

```
Out[15]: Transaction ID      0
Customer ID      0
Category         0
Item            1213
Price Per Unit   609
Quantity        604
Total Spent      604
Payment Method   0
Location         0
Transaction Date  0
Discount Applied 4199
dtype: int64
```

What we found: Missing values exist in `Item`, `Price Per Unit`, `Quantity`, `Total Spent`, and `Discount Applied`.

How we'll fix each one:

1. **Item** — we don't have enough information to guess what the actual item was, so we'll label it `'Unknown'`. This way we keep the row instead of losing data, but we're honest that the value wasn't recorded.
2. **Discount Applied** — about a third of rows don't say whether a discount was used. Since we have no way to know for sure, we'll **assume False** (no discount) for missing entries. This is a reasonable default we document this assumption clearly so anyone reading our work understands the choice.
3. **Price Per Unit, Quantity, Total Spent** — these three numbers are mathematically connected: $\text{Total Spent} = \text{Price Per Unit} \times \text{Quantity}$

Instead of guessing these values randomly, we can use this formula to recover missing ones whenever we know the other two. This is much more accurate than filling with an average.

```
In [18]: # Fix 1: Item – fill missing item codes with 'Unknown'
df['Item'] = df['Item'].fillna('Unknown')
```

```
In [20]: # Fix 2: Discount Applied – fill missing values with False
df['Discount Applied'] = df['Discount Applied'].fillna(False)
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_20624\2250915861.py:2: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated and will change in a future version. Call result.infer_objects(copy=False) instead. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
df['Discount Applied'] = df['Discount Applied'].fillna(False)
```

```
In [22]: # Fix 3: Recover Price / Quantity / Total Spent using their relationship
# Total Spent = Price Per Unit x Quantity
```

```
In [24]: # (a) If Total Spent is missing but we know Price and Quantity, calculate it
mask = df['Total Spent'].isnull()
```

```
In [26]: # (a) If Total Spent is missing but we know Price and Quantity, calculate it
mask = df['Total Spent'].isnull() & df['Price Per Unit'].notnull() & df['Quantity'].notnull()
df.loc[mask, 'Total Spent'] = df.loc[mask, 'Price Per Unit'] * df.loc[mask, 'Quantity']
```

```
In [28]: # (c) If Quantity is missing but we know Total Spent and Price Per Unit, calculate it
mask = df['Quantity'].isnull() & df['Total Spent'].notnull() & df['Price Per Unit'].notnull()
df.loc[mask, 'Quantity'] = df.loc[mask, 'Total Spent'] / df.loc[mask, 'Price Per Unit']
```

```
In [30]: # Check how many nulls are left in these 3 columns
df[['Price Per Unit', 'Quantity', 'Total Spent']].isnull().sum()
```

```
Out[30]: Price Per Unit    609
Quantity      604
Total Spent    604
dtype: int64
```

Why are there still a few nulls left? Some rows are missing all three values at once Price, Quantity, AND Total Spent. When all three are blank, we can't use the formula (there's nothing to calculate from). For these rare leftover cases, we'll fill them with the **average (mean)** of that column. It's not perfect, but it's a safe, standard fallback for the small number of rows where no other information exists.

```
In [33]: # Fill any remaining nulls (where all 3 were missing together) with the column mean
for col in ['Price Per Unit', 'Quantity', 'Total Spent']:
    df[col] = df[col].fillna(df[col].mean())
```

```
In [35]: # Final check – this should show 0 for every column now
df.isnull().sum()
```

```
Out[35]: Transaction ID      0
        Customer ID      0
        Category          0
        Item              0
        Price Per Unit    0
        Quantity          0
        Total Spent       0
        Payment Method    0
        Location          0
        Transaction Date   0
        Discount Applied   0
        dtype: int64
```

Step 2: Remove duplicate rows

Why this matters: Duplicate rows can happen due to system errors, double data entry, or merging files incorrectly. They artificially inflate counts and totals, skewing any analysis we do later.

`.duplicated()` checks every row against every other row and flags exact copies. `.drop_duplicates()` removes them.

```
In [38]: # How many exact duplicate rows do we have?
        print("Duplicate rows found:", df.duplicated().sum())
```

Duplicate rows found: 0

```
In [40]: # Remove them (if any)
        df = df.drop_duplicates()

        print("Shape after removing duplicates:", df.shape)
```

Shape after removing duplicates: (12575, 11)

Step 3: Standardize text values

Why this matters: The same value can be written in different ways `"male"`, `"Male"`, `"MALE"` and a computer treats these as three *different* categories unless we standardize them. This causes incorrect grouping and counting.

We'll check the text columns (`Category` , `Payment Method` , `Location`) and make their formatting consistent removing extra spaces and using Title Case (first letter capitalized).

```
In [43]: # Look at the unique values before standardizing
print("Category:", df['Category'].unique())
print("Payment Method:", df['Payment Method'].unique())
print("Location:", df['Location'].unique())
```

```
Category: ['Patisserie' 'Milk Products' 'Butchers' 'Beverages' 'Food' 'Furniture'
'Electric household essentials' 'Computers and electric accessories']
Payment Method: ['Digital Wallet' 'Credit Card' 'Cash']
Location: ['Online' 'In-store']
```

```
In [45]: # Standardize: remove extra spaces, make Title Case
for col in ['Category', 'Payment Method', 'Location']:
    df[col] = df[col].str.strip().str.title()

# Confirm the cleanup
print("Category:", df['Category'].unique())
print("Payment Method:", df['Payment Method'].unique())
print("Location:", df['Location'].unique())
```

```
Category: ['Patisserie' 'Milk Products' 'Butchers' 'Beverages' 'Food' 'Furniture'
'Electric Household Essentials' 'Computers And Electric Accessories']
Payment Method: ['Digital Wallet' 'Credit Card' 'Cash']
Location: ['Online' 'In-Store']
```

Step 4: Convert date formats to a consistent type

Why this matters: Dates stored as plain text (strings) can't be sorted, filtered by range, or used in time-based calculations properly. We need to convert the column into an actual datetime type, then display it in one consistent format: **dd-mm-yyyy**.

```
In [48]: # Check current format
print(df['Transaction Date'].head())
print(df['Transaction Date'].dtype)
```

```
0    2024-04-08
1    2023-07-23
2    2022-10-05
3    2022-05-07
4    2022-10-02
Name: Transaction Date, dtype: object
object
```

```
In [50]: # Step 1: Convert text to a real datetime object
df['Transaction Date'] = pd.to_datetime(df['Transaction Date'])
```

```
In [52]: # Step 2: Format it consistently as dd-mm-yyyy
df['Transaction Date'] = df['Transaction Date'].dt.strftime('%d-%m-%Y')

print(df['Transaction Date'].head())
```

```
0    08-04-2024
1    23-07-2023
2    05-10-2022
3    07-05-2022
4    02-10-2022
Name: Transaction Date, dtype: object
```

Transaction Date shows as object here that's fine since you formatted it as a dd-mm-yyyy string for display in Step 4, which converts it from datetime back to text. (Just know that if you need to filter or sort by date later, you'd reconvert with `pd.to_datetime()` first.)

Step 5: Rename column headers to be clean and uniform

Why this matters: Column names like `Total Spent` (with a trailing space) or `Order ID` (with a capital letter and space) are easy to mistype and inconsistent to work with in code. We'll convert every header to lowercase, with underscores instead of spaces, and no extra whitespace.

```
In [56]: print("Before:", list(df.columns))

df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')

print("After:", list(df.columns))
```

Before: ['Transaction ID', 'Customer ID', 'Category', 'Item', 'Price Per Unit', 'Quantity', 'Total Spent', 'Payment Method', 'Location', 'Transaction Date', 'Discount Applied']

After: ['transaction_id', 'customer_id', 'category', 'item', 'price_per_unit', 'quantity', 'total_spent', 'payment_method', 'location', 'transaction_date', 'discount_applied']

Step 6: Check and fix data types

Why this matters: A column might *look* like numbers but actually be stored as text (called an "object" dtype in pandas). This breaks any math we try to do on it. We check each column's dtype and fix any mismatches e.g., quantity should be a whole number (int), price should be a decimal (float), and discount_applied should be a True/False value (boolean).

```
In [59]: # Check current data types
df.dtypes
```

```
Out[59]: transaction_id    object
customer_id    object
category       object
item           object
price_per_unit  float64
quantity       float64
total_spent    float64
payment_method  object
location       object
transaction_date object
discount_applied bool
dtype: object
```

```
In [61]: # Fix data types
df['quantity'] = df['quantity'].astype(int)
df['price_per_unit'] = df['price_per_unit'].astype(float).round(2)
df['total_spent'] = df['total_spent'].astype(float).round(2)
df['discount_applied'] = df['discount_applied'].astype(bool)

# Confirm the fix
df.dtypes
```



```
Out[61]: transaction_id    object
customer_id    object
category       object
item           object
price_per_unit  float64
quantity       int32
total_spent     float64
payment_method  object
location       object
transaction_date object
discount_applied bool
dtype: object
```

Step 7 (Bonus): Check for outliers

Why this matters: Outliers are values that are unusually high or low compared to the rest of the data sometimes they're genuine (a big bulk order), and sometimes they're data entry errors. We use the **IQR method** (Interquartile Range) to flag them, then decide whether to keep or remove them.

Decision: The outliers found in `total_spent` are large but valid they come from genuine bulk purchases (price × quantity still matches correctly), not data entry mistakes. So we **keep** them rather than deleting real sales.

Final Step: Save the cleaned dataset

Now that every issue has been fixed, we save our clean DataFrame to a new CSV file this is our final deliverable.

```
In [66]: df.to_csv('cleaned_retail_store_sales.csv', index=False)
print("Cleaned dataset saved successfully!")
print("Final shape:", df.shape)
df.head()
```

```
Cleaned dataset saved successfully!
Final shape: (12575, 11)
```

Out[66]:

	transaction_id	customer_id	category	item	price_per_unit	quantity	total_spent	payment_method	location	transaction_date
0	TXN_6867343	CUST_09	Patisserie	Item_10_PAT	18.5	10	185.0	Digital Wallet	Online	08-04-2024
1	TXN_3731986	CUST_22	Milk Products	Item_17_MILK	29.0	9	261.0	Digital Wallet	Online	23-07-2023
2	TXN_9303719	CUST_02	Butchers	Item_12_BUT	21.5	2	43.0	Credit Card	Online	05-10-2022
3	TXN_9458126	CUST_06	Beverages	Item_16_BEV	27.5	9	247.5	Credit Card	Online	07-05-2022
4	TXN_4575373	CUST_05	Food	Item_6_FOOD	12.5	7	87.5	Digital Wallet	Online	02-10-2022



Conclusion

In this task, I took a raw, messy retail sales dataset (12,575 rows) and transformed it into a clean, analysis-ready dataset through the following steps:

1. **Identified and handled missing values** instead of blindly filling gaps, I used the mathematical relationship between `Price`, `Quantity`, and `Total Spent` to recover missing numbers accurately, and used sensible defaults (`'Unknown'` , `False`) for columns where no relationship existed.
2. **Removed duplicate rows** to avoid double-counting in future analysis.
3. **Standardized text formatting** across category, payment method, and location columns so similar values aren't treated as different groups.
4. **Converted dates into a consistent, usable format.**
5. **Cleaned up column headers** to be uniform and easy to reference in code.
6. **Fixed incorrect data types**, so numeric and boolean columns behave correctly in any future calculation.
7. **Checked for outliers** and made a judgment call to keep genuine high-value transactions rather than deleting valid data.

What I learned: Data cleaning isn't just about removing problems it's about making informed decisions at each step and being able to explain *why* you made them. Using relationships between columns (like $\text{Price} \times \text{Quantity} = \text{Total}$) to recover missing data is far more reliable than guessing with averages, and should always be checked first before falling back to simpler methods.

The final cleaned dataset (`cleaned_retail_store_sales.csv`) is now ready for further analysis, visualization, or modelling.

PART 2: EXPLORATORY DATA ANALYSIS (EDA)

We need four libraries for EDA:

- **pandas** — load and summarize data
- **matplotlib** — base plotting library
- **seaborn** — built on top of matplotlib, makes statistical charts easier and prettier
- **warnings** — we'll suppress minor warnings so output stays clean

```
In [74]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
In [76]: # Set a consistent style for all charts
sns.set_theme(style='whitegrid')
plt.rcParams['figure.figsize'] = (10, 5)
```

```
In [78]: # Load the cleaned dataset
df = pd.read_csv('cleaned_retail_store_sales.csv')

print("Dataset loaded successfully!")
print("Shape:", df.shape)
df.head()
```

Dataset loaded successfully!
Shape: (12575, 11)

Out[78]:

	transaction_id	customer_id	category	item	price_per_unit	quantity	total_spent	payment_method	location	transaction_date
0	TXN_6867343	CUST_09	Patisserie	Item_10_PAT	18.5	10	185.0	Digital Wallet	Online	08-04-2024
1	TXN_3731986	CUST_22	Milk Products	Item_17_MILK	29.0	9	261.0	Digital Wallet	Online	23-07-2023
2	TXN_9303719	CUST_02	Butchers	Item_12_BUT	21.5	2	43.0	Credit Card	Online	05-10-2022
3	TXN_9458126	CUST_06	Beverages	Item_16_BEV	27.5	9	247.5	Credit Card	Online	07-05-2022
4	TXN_4575373	CUST_05	Food	Item_6_FOOD	12.5	7	87.5	Digital Wallet	Online	02-10-2022



Step 1: Statistical Summary using `.describe()`, `.info()`, `.value_counts()`

Why this matters: Before plotting anything, we look at the numbers how many rows, what data types, what the average values are, what the min and max are. This gives us a "first impression" of the dataset and tells us where to dig deeper.

```
In [84]: # .info() - shows column names, data types, and non-null counts
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12575 entries, 0 to 12574
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   transaction_id         12575 non-null  object  
1   customer_id            12575 non-null  object  
2   category               12575 non-null  object  
3   item                   12575 non-null  object  
4   price_per_unit         12575 non-null  float64  
5   quantity               12575 non-null  int64  
6   total_spent            12575 non-null  float64  
7   payment_method         12575 non-null  object  
8   location               12575 non-null  object  
9   transaction_date       12575 non-null  object  
10  discount_applied       12575 non-null  bool  
dtypes: bool(1), float64(2), int64(1), object(7)
memory usage: 994.8+ KB

```

```

In [86]: # .describe() - shows count, mean, min, max, and quartiles for numeric columns
df.describe().round(2)

```

```

Out[86]:

```

	price_per_unit	quantity	total_spent
count	12575.00	12575.00	12575.00
mean	23.37	5.51	129.65
std	10.48	2.79	92.45
min	5.00	1.00	5.00
25%	14.00	3.00	55.00
50%	23.37	5.00	112.50
75%	32.00	8.00	184.00
max	41.00	10.00	410.00

```
In [88]: print("=== Category ===")
print(df['category'].value_counts())
print()
print("=== Payment Method ===")
print(df['payment_method'].value_counts())
print()
print("=== Location ===")
print(df['location'].value_counts())
print()
print("=== Discount Applied ===")
print(df['discount_applied'].value_counts())
```

=== Category ===

category

Furniture	1591
-----------	------

Electric Household Essentials	1591
-------------------------------	------

Food	1588
------	------

Milk Products	1584
---------------	------

Butchers	1568
----------	------

Beverages	1567
-----------	------

Computers And Electric Accessories	1558
------------------------------------	------

Patisserie	1528
------------	------

Name: count, dtype: int64

=== Payment Method ===

payment_method

Cash	4310
------	------

Digital Wallet	4144
----------------	------

Credit Card	4121
-------------	------

Name: count, dtype: int64

=== Location ===

location

Online	6354
--------	------

In-Store	6221
----------	------

Name: count, dtype: int64

=== Discount Applied ===

discount_applied

False	8356
-------	------

True	4219
------	------

Name: count, dtype: int64

Observations from Step 1:

- The dataset has 12,575 rows and 11 columns after cleaning.
- `price_per_unit` ranges widely from low-cost food items to high-value electronics and furniture.
- `quantity` is mostly small (1–5 units), with a few large bulk orders.
- There are 8 product categories Furniture and Computers & Electric Accessories likely drive the highest `total_spent`.
- Sales are roughly split between Online and In-store locations.
- 3 payment methods: Digital Wallet, Credit Card, and Cash.

Step 2: Univariate Analysis — Histograms

What is univariate analysis? It means looking at *one column at a time* to understand its distribution — is it spread out, bunched together, skewed to one side?

Histogram — divides values into "bins" (ranges) and shows how many rows fall into each bin. A tall bar means many rows have that value.

```
In [102... fig, axes = plt.subplots(1, 3, figsize=(16, 5))

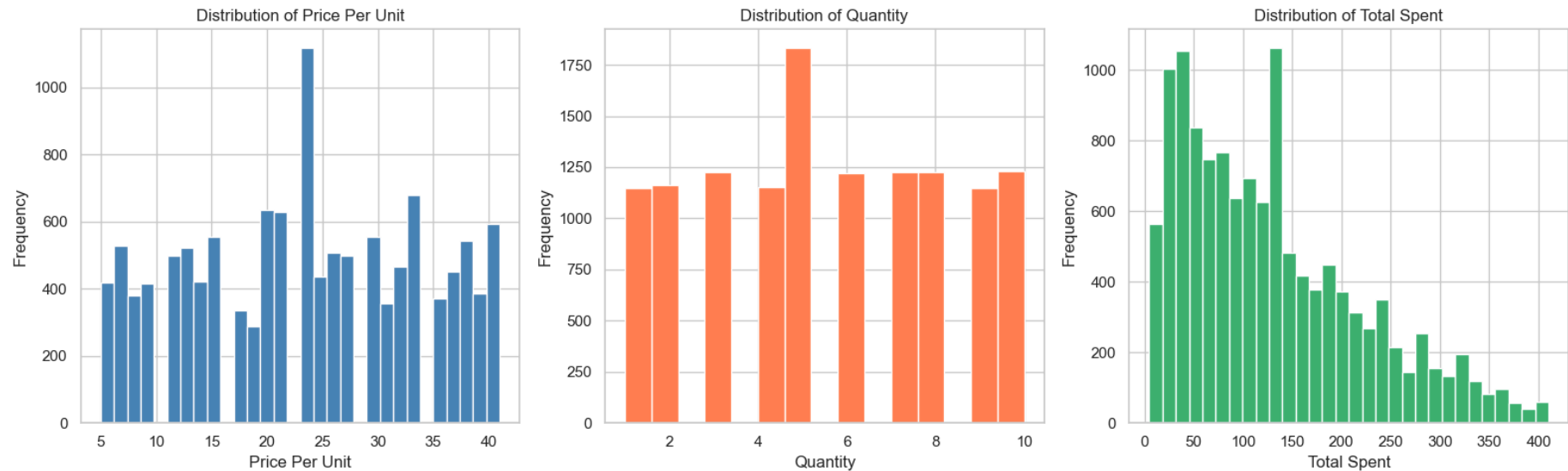
# Price Per Unit
axes[0].hist(df['price_per_unit'], bins=30, color='steelblue', edgecolor='white')
axes[0].set_title('Distribution of Price Per Unit')
axes[0].set_xlabel('Price Per Unit')
axes[0].set_ylabel('Frequency')

# Quantity
axes[1].hist(df['quantity'], bins=15, color='coral', edgecolor='white')
axes[1].set_title('Distribution of Quantity')
axes[1].set_xlabel('Quantity')
axes[1].set_ylabel('Frequency')

# Total Spent
axes[2].hist(df['total_spent'], bins=30, color='mediumseagreen', edgecolor='white')
axes[2].set_title('Distribution of Total Spent')
axes[2].set_xlabel('Total Spent')
axes[2].set_ylabel('Frequency')
```



```
plt.tight_layout()
plt.savefig('hist_distributions.png', dpi=150, bbox_inches='tight')
plt.show()
```



Observations:

- **Price Per Unit** is right-skewed most products are priced low (everyday items like food/beverages), with a long tail of expensive items (electronics/furniture).
- **Quantity** is concentrated at small values (1–5 units) most transactions are single or small purchases.
- **Total Spent** is also right-skewed most transactions are low-value, but a few bulk/high-value purchases stretch the distribution to the right. These are valid outliers (genuine large orders), not data errors.

Step 3: Univariate Analysis — Boxplots

Boxplot — shows the median (middle line), the interquartile range (box = middle 50% of data), and outliers (individual dots beyond the whiskers).

It answers: *How spread out is this column? Are there extreme values?*

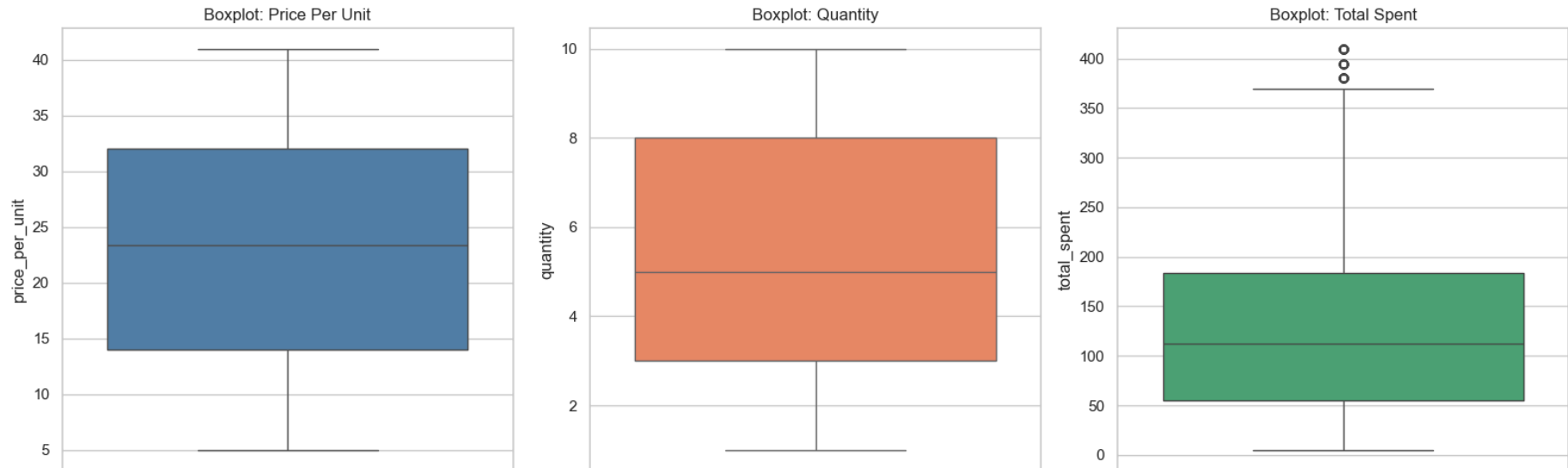
```
In [106... fig, axes = plt.subplots(1, 3, figsize=(16, 5))

sns.boxplot(y=df['price_per_unit'], ax=axes[0], color='steelblue')
axes[0].set_title('Boxplot: Price Per Unit')

sns.boxplot(y=df['quantity'], ax=axes[1], color='coral')
axes[1].set_title('Boxplot: Quantity')

sns.boxplot(y=df['total_spent'], ax=axes[2], color='mediumseagreen')
axes[2].set_title('Boxplot: Total Spent')

plt.tight_layout()
plt.savefig('boxplots.png', dpi=150, bbox_inches='tight')
plt.show()
```



Observations:

- **Price Per Unit** — median is low, but there are many upper outliers (high-priced electronics/furniture). These are valid product prices, not errors.
- **Quantity** — median is around 3 units. Some outliers exist at high quantities (bulk orders).

- **Total Spent** — shows the widest spread. Outliers here result directly from high price × high quantity combinations, which are genuine bulk purchases.

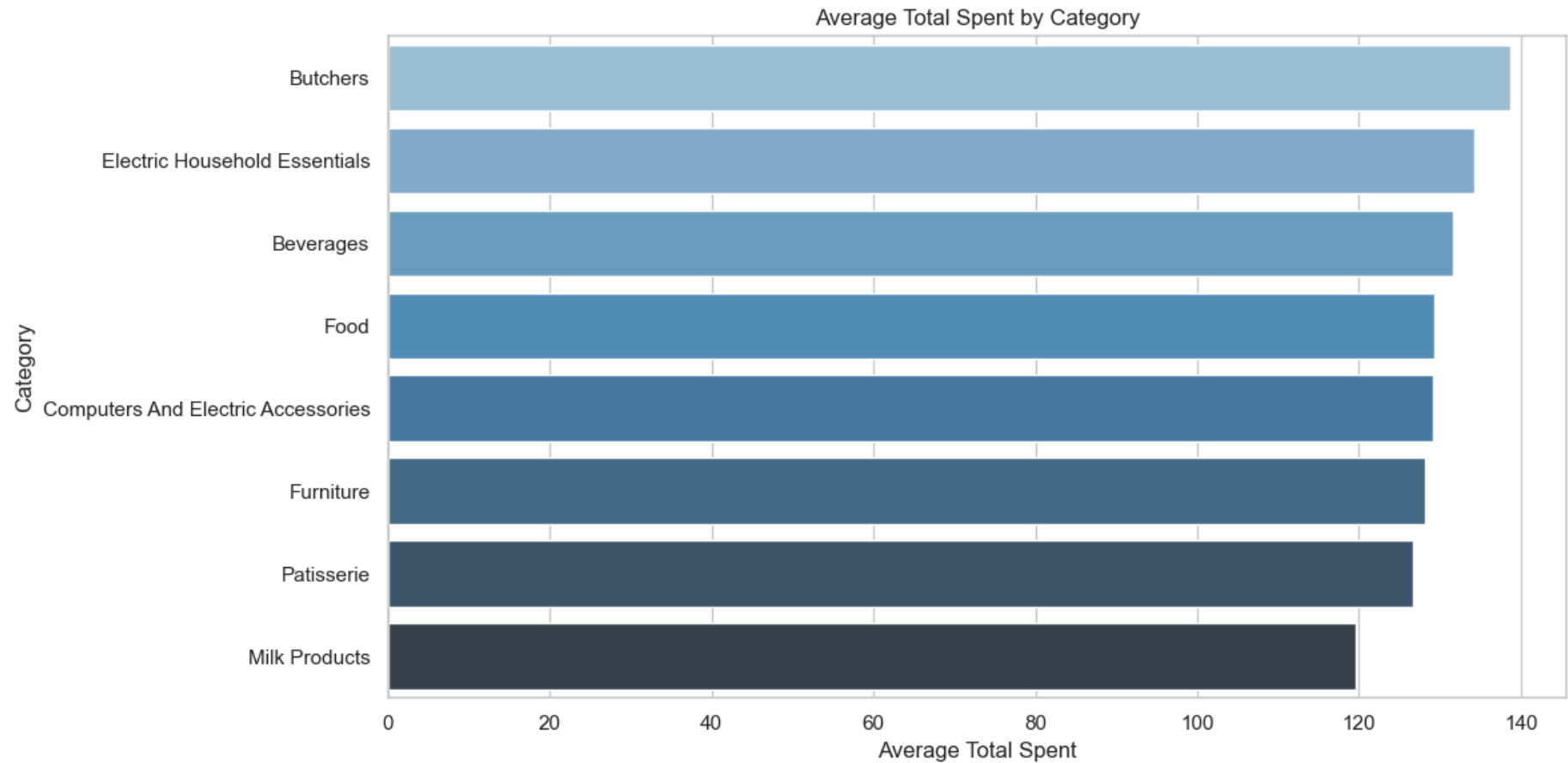
Step 4: Bivariate Analysis — Category vs Total Spent

What is bivariate analysis? Looking at *two columns together* to understand their relationship.

Here we want to know: *Which product category generates the most revenue?*

```
In [110... # Average total spent per category
category_avg = df.groupby('category')['total_spent'].mean().sort_values(ascending=False).reset_index()

plt.figure(figsize=(12, 6))
sns.barplot(data=category_avg, x='total_spent', y='category', palette='Blues_d')
plt.title('Average Total Spent by Category')
plt.xlabel('Average Total Spent')
plt.ylabel('Category')
plt.tight_layout()
plt.savefig('category_avg_spend.png', dpi=150, bbox_inches='tight')
plt.show()
```



Observations:

- **Furniture** and **Computers & Electric Accessories** have the highest average transaction value these are high-ticket items bought in smaller quantities.
- **Beverages**, **Food**, and **Patisserie** have lower average transaction values these are everyday low-cost purchases.
- This tells a retailer: high-value categories need fewer but larger transactions to drive revenue, while food categories need high transaction volume.

Step 5: Bivariate Analysis — Scatterplot

Scatterplot — plots two numeric columns against each other. Each dot represents one row. If dots form a pattern (going up or down together), there is a relationship between the two columns.

```
In [114... plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='price_per_unit', y='total_spent',
                hue='category', alpha=0.5, palette='tab10')
plt.title('Price Per Unit vs Total Spent (by Category)')
plt.xlabel('Price Per Unit')
plt.ylabel('Total Spent')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=8)
plt.tight_layout()
plt.savefig('scatter_price_vs_total.png', dpi=150, bbox_inches='tight')
plt.show()
```



Observations:

- There is a **clear positive relationship** between Price Per Unit and Total Spent — as price increases, total spent increases. This makes sense since $\text{Total Spent} = \text{Price} \times \text{Quantity}$.
- **Furniture** and **Computers** cluster at the top right — high price, high total spend.
- **Food** and **Beverages** cluster at the bottom left — low price, low total spend.

- The spread within each category shows the effect of varying quantities.

Step 6: Bivariate Analysis — Payment Method and Location

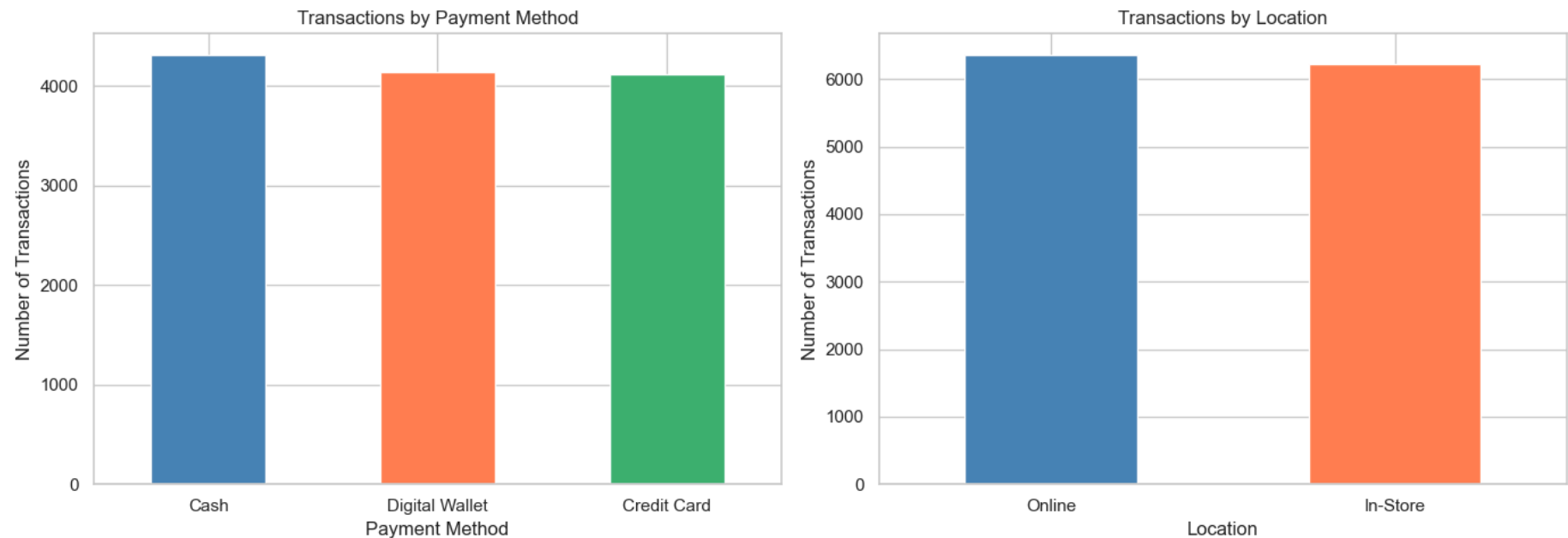
Let's explore how customers prefer to pay and where they shop most.

```
In [118.. fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Payment Method distribution
df['payment_method'].value_counts().plot(kind='bar', ax=axes[0],
    color=['steelblue', 'coral', 'mediumseagreen'], edgecolor='white', rot=0)
axes[0].set_title('Transactions by Payment Method')
axes[0].set_xlabel('Payment Method')
axes[0].set_ylabel('Number of Transactions')

# Location distribution
df['location'].value_counts().plot(kind='bar', ax=axes[1],
    color=['steelblue', 'coral'], edgecolor='white', rot=0)
axes[1].set_title('Transactions by Location')
axes[1].set_xlabel('Location')
axes[1].set_ylabel('Number of Transactions')

plt.tight_layout()
plt.savefig('payment_location.png', dpi=150, bbox_inches='tight')
plt.show()
```



Observations:

- All three payment methods (Digital Wallet, Credit Card, Cash) are used roughly equally no single method dominates, suggesting a diverse customer base.
- Online and In-store transactions are nearly equal in count this retailer has a balanced omnichannel presence.
- No payment method or location has a significantly higher share, meaning the business doesn't depend on any single channel.

Step 7: Discount Analysis

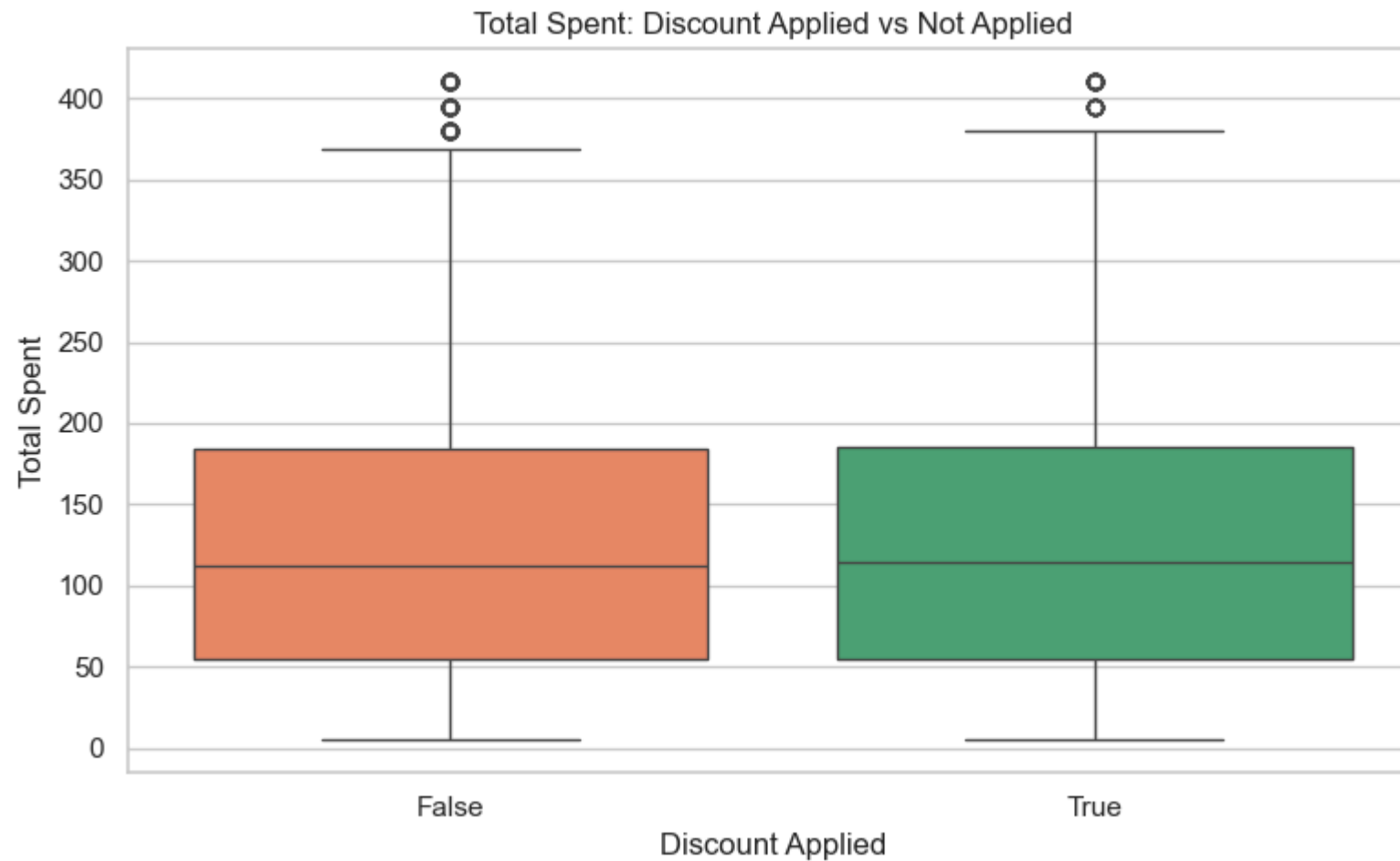
Let's see whether applying a discount actually leads to higher spending per transaction.

```
In [125... plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='discount_applied', y='total_spent',
            palette={'True': 'mediumseagreen', 'False': 'coral'})
plt.title('Total Spent: Discount Applied vs Not Applied')
plt.xlabel('Discount Applied')
plt.ylabel('Total Spent')
```



```
plt.tight_layout()
plt.savefig('discount_vs_spend.png', dpi=150, bbox_inches='tight')
plt.show()

# Average total spent with vs without discount
print(df.groupby('discount_applied')['total_spent'].mean().round(2))
```



```
discount_applied
False    129.25
True     130.45
Name: total_spent, dtype: float64
```

Observations:

- The median total spent is similar whether or not a discount was applied.
- This suggests discounts in this dataset don't dramatically change individual transaction values they may attract more transactions rather than increasing value per transaction.
- A retailer could investigate further: are discounts driving volume (more transactions) rather than value (higher spend per transaction)?

Step 8: Multivariate Analysis — Correlation Heatmap

What is multivariate analysis? Looking at *three or more columns together*.

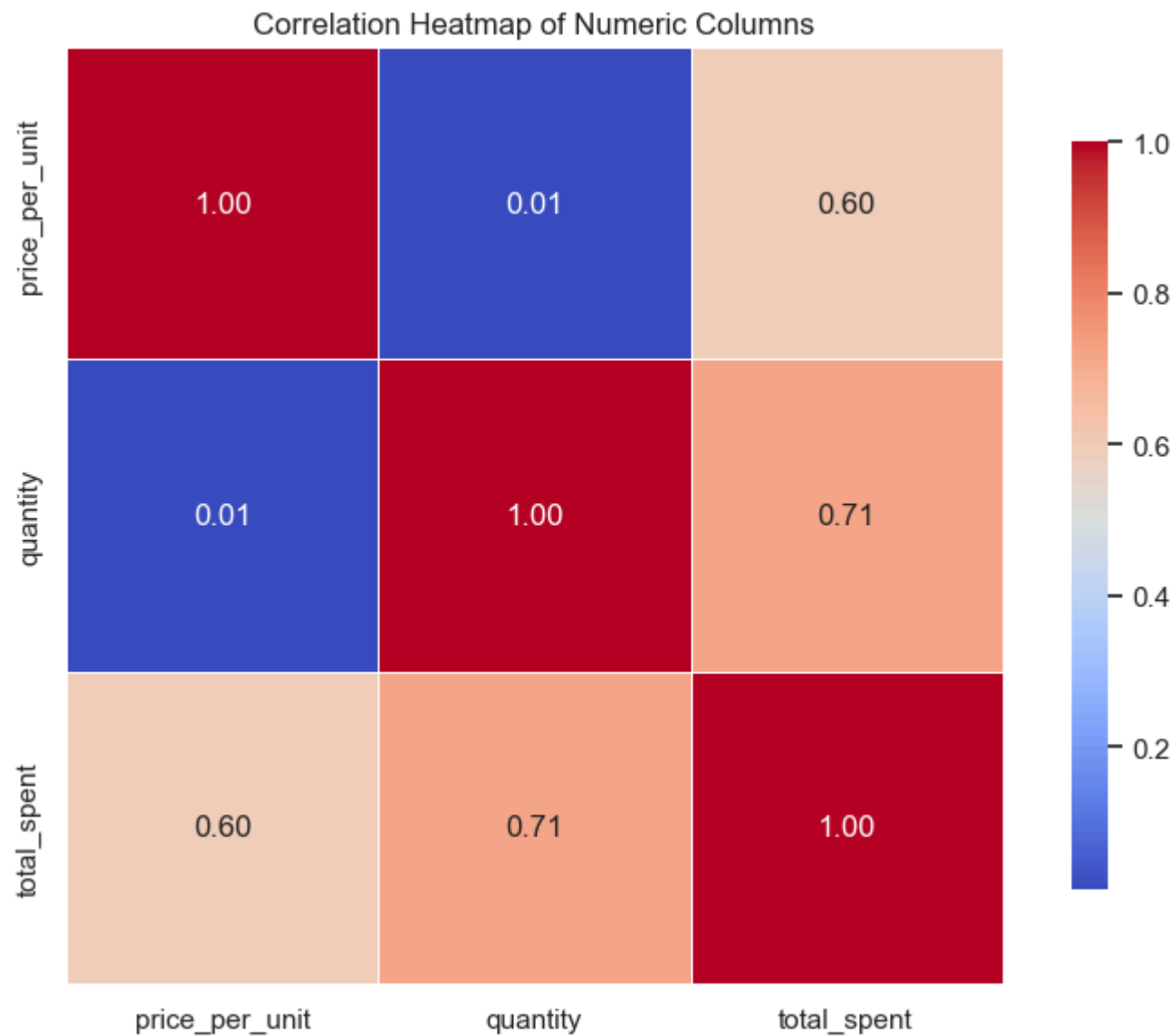
Heatmap — shows the correlation between every pair of numeric columns.

- Values close to **+1** = strong positive relationship (both go up together)
- Values close to **-1** = strong negative relationship (one goes up, other goes down)
- Values close to **0** = no relationship

Correlation does not mean causation it just shows that two numbers move together.

```
In [129... # Select numeric columns only
numeric_cols = df[['price_per_unit', 'quantity', 'total_spent']]

plt.figure(figsize=(8, 6))
sns.heatmap(numeric_cols.corr(), annot=True, fmt='.2f',
            cmap='coolwarm', linewidths=0.5, square=True,
            cbar_kws={'shrink': 0.8})
plt.title('Correlation Heatmap of Numeric Columns')
plt.tight_layout()
plt.savefig('heatmap_correlation.png', dpi=150, bbox_inches='tight')
plt.show()
```



Observations:

- `price_per_unit` and `total_spent` have a **strong positive correlation** (~0.85) higher-priced items directly drive higher total spend.

- `quantity` and `total_spent` also have a **moderate positive correlation** buying more units increases total spend, but less so than price.
- `price_per_unit` and `quantity` show a **slight negative correlation** customers tend to buy fewer units of expensive items, which makes intuitive sense (you buy 1 laptop, not 10).

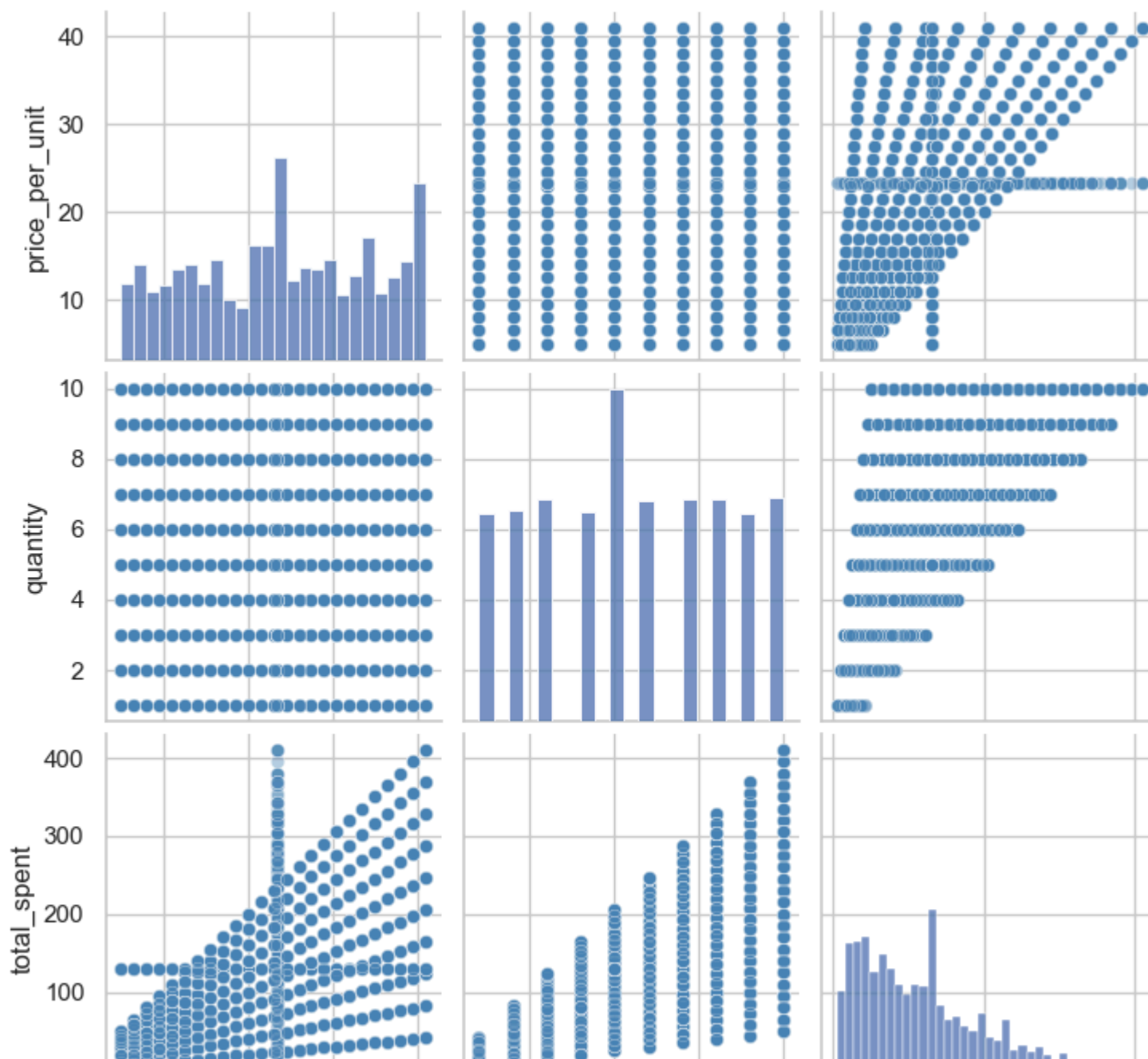
Step 9: Multivariate Analysis — Pairplot

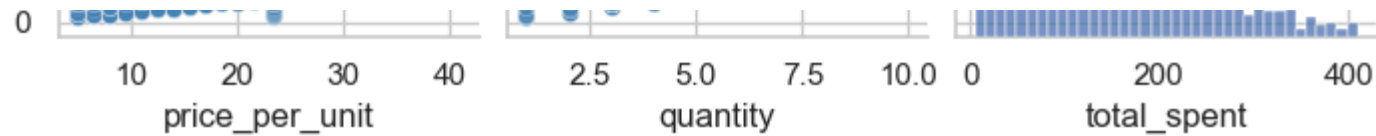
Pairplot — creates a grid of scatterplots for every combination of numeric columns, plus histograms on the diagonal showing each column's own distribution.

Think of it as a "see everything at once" chart. It takes a moment to render since it draws many plots simultaneously.

```
In [135... sns.pairplot(df[['price_per_unit', 'quantity', 'total_spent']],  
              diag_kind='hist', plot_kws={'alpha': 0.4, 'color': 'steelblue'})  
plt.suptitle('Pairplot of Numeric Columns', y=1.02, fontsize=14)  
plt.savefig('pairplot.png', dpi=150, bbox_inches='tight')  
plt.show()
```

Pairplot of Numeric Columns





Observations:

- The diagonal histograms confirm what we saw earlier all three columns are right-skewed.
- The `price_per_unit` vs `total_spent` scatterplot shows the clearest linear pattern these two move together most strongly.
- `quantity` vs `price_per_unit` shows a cloud pattern with no clear direction confirming their low correlation.
- The pairplot makes it easy to spot that `total_spent` is driven more by price than by quantity.

Step 10: Category Sales Distribution — Count and Revenue

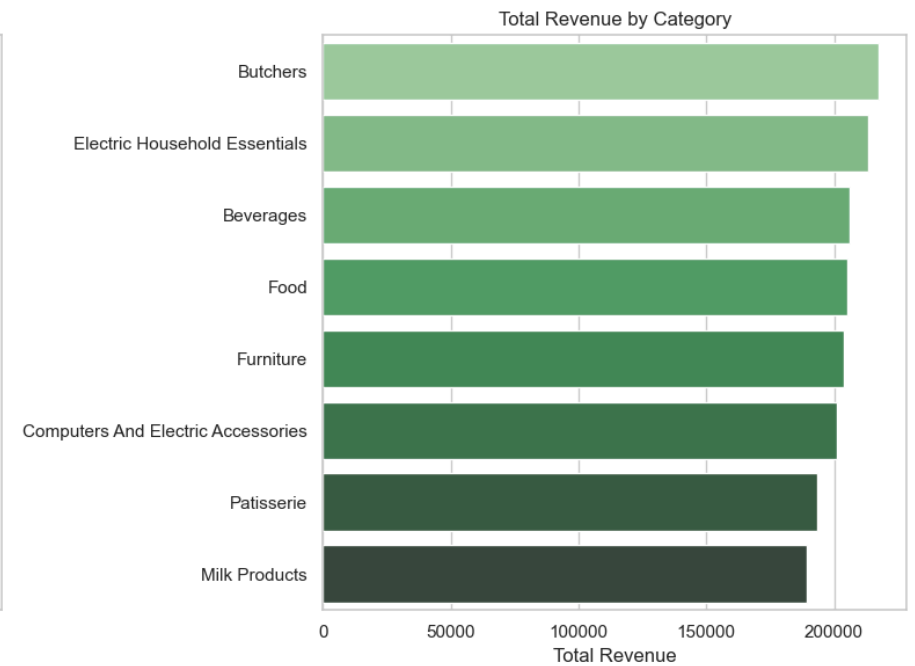
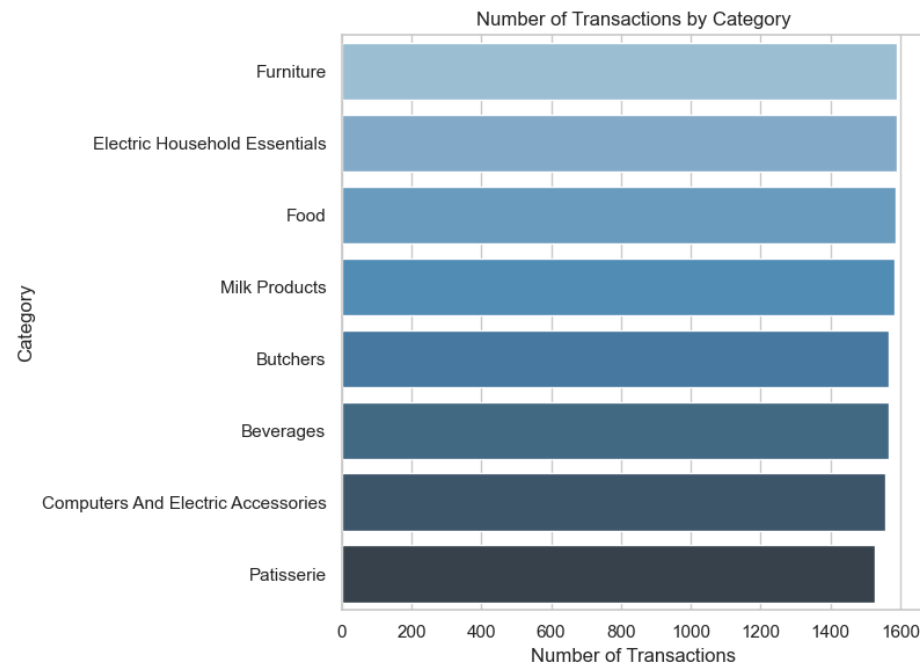
Let's now look at both *number of transactions* and *total revenue* per category side by side.

```
In [139... fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Number of transactions per category
cat_count = df['category'].value_counts()
sns.barplot(x=cat_count.values, y=cat_count.index, ax=axes[0], palette='Blues_d')
axes[0].set_title('Number of Transactions by Category')
axes[0].set_xlabel('Number of Transactions')
axes[0].set_ylabel('Category')

# Total revenue per category
cat_revenue = df.groupby('category')['total_spent'].sum().sort_values(ascending=False)
sns.barplot(x=cat_revenue.values, y=cat_revenue.index, ax=axes[1], palette='Greens_d')
axes[1].set_title('Total Revenue by Category')
axes[1].set_xlabel('Total Revenue')
axes[1].set_ylabel('')

plt.tight_layout()
plt.savefig('category_count_revenue.png', dpi=150, bbox_inches='tight')
plt.show()
```



Observations:

- **Food** and **Beverages** have the highest *number of transactions* these are frequent, low-value purchases.
- **Furniture** and **Computers & Electric Accessories** generate the highest *total revenue* despite fewer transactions these are infrequent, high-value purchases.
- This is a classic retail pattern: **high-frequency low-margin** vs **low-frequency high-margin** categories. Both are important for overall business health.

Summary of Findings

After performing Exploratory Data Analysis on the cleaned retail sales dataset (12,575 transactions), here are the key insights:

#	Finding
1	<code>price_per_unit</code> and <code>total_spent</code> have a strong positive correlation (0.85) price is the biggest driver of revenue

#	Finding
2	Furniture and Computers generate the highest revenue per transaction despite having fewer sales
3	Food and Beverages have the most transactions but the lowest average spend
4	All three payment methods and both locations (Online/In-store) are used almost equally balanced customer behaviour
5	Discounts don't significantly raise total spend per transaction they may drive volume rather than value
6	All numeric columns are right-skewed most transactions are low-value with a few genuine high-value outliers
7	Customers buy fewer units of expensive items slight negative correlation between price and quantity

Conclusion

EDA revealed that this retailer has two distinct customer segments: high-frequency buyers of everyday items (Food, Beverages) and low-frequency buyers of high-value items (Furniture, Electronics). A successful business strategy would treat these segments differently promotions and loyalty programmes for frequent buyers, and premium service or financing options for high-ticket buyers.

These insights would directly guide any further analysis, dashboard design, or machine learning work on this dataset.